

Brain Tumor Classification from MRI Images

Pituitary Tumor vs. No Tumor using Transfer Learning (ResNet18) and a Desktop
Prediction GUI

Framework	PyTorch / torchvision
Model	ResNet18 (ImageNet pre-trained) with custom classification head
Task	Binary image classification (no_tumor, pituitary_tumor)
Test accuracy	91.06% (163 / 179 images)
Deliverables	Training script, trained weights, Tkinter GUI application

Abstract

This project develops a deep learning system that classifies brain MRI images into two categories: **pituitary tumor** and **no tumor**. A ResNet18 network pre-trained on ImageNet is used as a frozen feature extractor, and a lightweight fully connected head is trained on the MRI data. Data augmentation (flips, rotation, brightness/contrast jitter) is applied to improve generalisation. After 20 epochs, the best model reaches roughly 91% accuracy on the held-out test set, with a precision of 91.4% and recall of 86.5% for the tumor class. The trained model is also wrapped in a desktop GUI so that a user can load an MRI image and receive a prediction with a confidence score.

1. Introduction

Brain tumors, including pituitary tumors, are typically diagnosed with the help of MRI. Reading these scans requires experienced radiologists and can be time consuming. Computer-aided diagnosis based on convolutional neural networks (CNNs) can act as a second reader, flagging suspicious scans and reducing workload. Because medical datasets are usually small, transfer learning from a large natural-image dataset is an effective strategy: the early layers already encode generic edges, textures and shapes.

2. Objectives

- Build a binary classifier that separates pituitary tumor MRI images from non-tumor images.
- Use transfer learning with ResNet18 to obtain strong accuracy with limited training time and data.
- Evaluate the model with accuracy, precision, recall, F1-score and a confusion matrix.
- Provide an easy-to-use GUI so that non-programmers can test the trained model.

3. Dataset

The data is organised in the standard ImageFolder layout, with a Training folder and a Testing folder, each containing one sub-folder per class (no_tumor, pituitary_tumor). The dataset is supplied as a zip archive that the script extracts automatically on first run. Class labels are assigned alphabetically by PyTorch: **0 = no_tumor**, **1 = pituitary_tumor**.

Split	Images	Notes
Training	Not shown in the supplied outputs	Augmentation applied, shuffled each epoch
Testing	179 (105 no tumor, 74 pituitary tumor)	Resize + normalise only; also used for validation

The test set is moderately imbalanced (about 59% no-tumor vs. 41% tumor), which is why per-class precision and recall are reported in addition to overall accuracy.

4. Methodology

4.1 Pre-processing and augmentation

All images are resized to 224 x 224 pixels and normalised with the ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225), matching the statistics the pre-trained backbone expects. Training images are additionally augmented:

- Random horizontal flip
- Random rotation of up to 15 degrees
- Colour jitter (brightness 0.2, contrast 0.2)

4.2 Model architecture

The backbone is **ResNet18** with ImageNet weights. All convolutional layers are frozen, so only the new classification head is trained. The original 1000-class layer is replaced with the following head:

Layer	Details
Dropout	p = 0.4
Linear	512 to 256
ReLU	Non-linearity
Dropout	p = 0.3
Linear	256 to 2 (class logits)

Freezing the backbone greatly reduces the number of trainable parameters, speeds up training, and lowers the risk of over-fitting on a small medical dataset. The two dropout layers add further regularisation.

4.3 Training configuration

Hyper-parameter	Value
Loss function	Cross-entropy
Optimizer	Adam (head parameters only), weight decay 1e-5
Learning rate	1e-4, StepLR (x0.1 every 7 epochs)
Batch size	32
Epochs	20
Model selection	Best accuracy epoch checkpoint saved as best_brain_tumor_model.pth
Device	CUDA GPU if available, otherwise CPU

5. Results

5.1 Training behaviour

Figure 1 shows the loss and accuracy curves. Both training and validation loss fall quickly during the first five epochs and then flatten. Training accuracy settles at roughly 93 to 95%, while validation accuracy plateaus at about 89 to 91%.

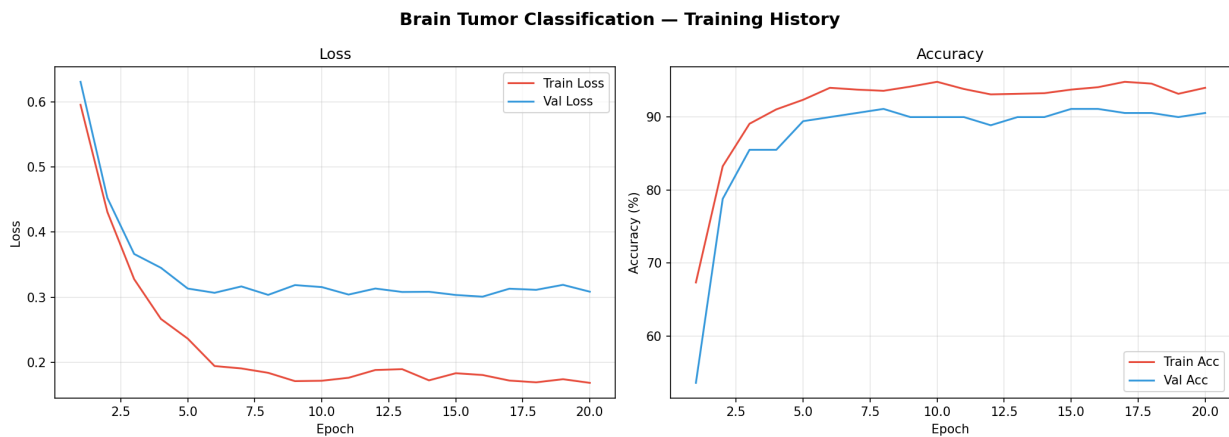


Figure 1. Training and validation loss (left) and accuracy (right) over 20 epochs.

5.2 Confusion matrix

The confusion matrix on the 179 test images is shown in Figure 2. Treating pituitary tumor as the positive class:

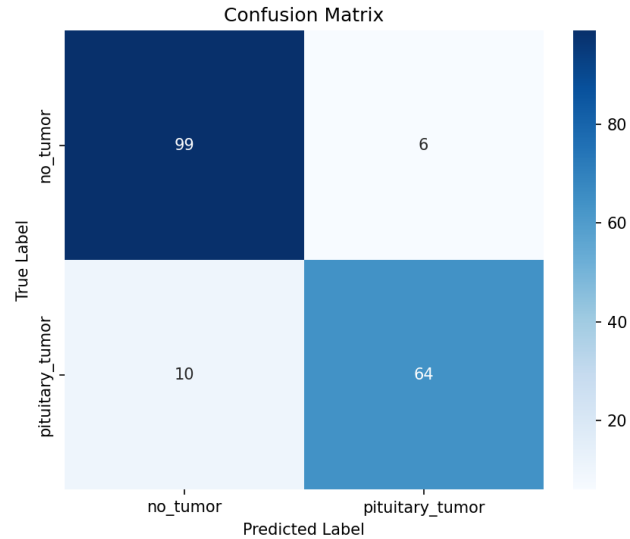


Figure 2. Confusion matrix on the test set.

	Predicted: no_tumor	Predicted: pituitary_tumor
True: no_tumor	99 (true negatives)	6 (false positives)
True: pituitary_tumor	10 (false negatives)	64 (true positives)

5.3 Performance metrics

Class	Precision	Recall	F1-score	Support
no_tumor	90.83%	94.29%	92.52%	105
pituitary_tumor	91.43%	86.49%	88.89%	74
Macro average	91.13%	90.39%	90.70%	179
Weighted average	91.08%	91.06%	91.02%	179
Overall accuracy			91.06%	179

Metrics are derived directly from the confusion matrix above (accuracy = 163 / 179). The reported sensitivity (recall) for the tumor class is 86.5% and the specificity is 94.3%.

6. Prediction GUI

A desktop application (`brain_tumor_gui.py`) built with Tkinter lets users run the trained model without writing code. The model is loaded in a background thread so the window stays responsive. The workflow is:

- Click **Choose MRI Image** and pick a JPG, PNG, BMP or TIFF file; a preview is displayed.
- Click **Analyze & Predict**. The image is resized, normalised and passed through the network using exactly the same test-time transforms as in training.
- The softmax output is converted into a diagnosis label (green for no tumor, red for pituitary tumor detected) together with an animated confidence bar and percentage.

The interface uses a dark theme and shows a status bar with the model-loading state, and it reports a clear error if the weights file is missing.

7. Discussion

The model reaches about 91% accuracy using only a trained classification head on top of a frozen backbone, which confirms that ImageNet features transfer reasonably well to MRI data. The gap between training accuracy (about 94%) and validation accuracy (about 90%), together with a validation loss that stays near 0.30 while training loss keeps falling, indicates mild over-fitting. Dropout and augmentation keep it under control, but further gains will need better adaptation to the medical domain.

The more important finding for a screening scenario is the error balance. The model produced 10 false negatives (missed tumors) against 6 false positives. Missing a tumor is clinically more costly than a false alarm, so improving tumor recall should be the priority in future iterations, for example by tuning the decision threshold or using class-weighted loss.

8. Limitations

- **Test set doubles as validation set.** The best epoch is chosen using test accuracy, so the reported figure is slightly optimistic. A separate validation split is recommended.
- **Small, single-source test set** (179 images), so the estimates carry noticeable uncertainty.
- **Binary scope only.** The model distinguishes pituitary tumor from no tumor and cannot recognise other tumor types or other pathologies.

- **Frozen backbone.** Features are not adapted to MRI texture, which limits the achievable accuracy.
- **Not a medical device.** The system is an educational prototype and must not be used for clinical diagnosis.

9. Future Work

- Fine-tune the last ResNet blocks with a lower learning rate, or try EfficientNet / DenseNet backbones.
- Create a proper train / validation / test split and use k-fold cross-validation.
- Extend to multi-class classification (glioma, meningioma, pituitary, no tumor).
- Add explainability such as Grad-CAM heat-maps so users can see which regions drive a prediction.
- Use class-weighted loss or threshold tuning to raise tumor recall; report ROC-AUC.
- Package the model as a web app or API for easier deployment.

10. Conclusion

This project demonstrates an end-to-end pipeline for MRI-based pituitary tumor detection: data loading and augmentation, transfer learning with ResNet18, quantitative evaluation, and an interactive GUI. With 91.06% test accuracy, 91.4% tumor precision and 86.5% tumor recall, the approach is a solid baseline. Fixing the validation protocol and fine-tuning the backbone are the most promising next steps toward a more reliable model.

Appendix: Software Environment

Component	Library
Deep learning	PyTorch, torchvision (ResNet18, ImageFolder, transforms)
Evaluation	scikit-learn (classification_report, confusion_matrix, accuracy_score)
Visualisation	matplotlib, seaborn
GUI	Tkinter, Pillow
Output files	best_brain_tumor_model.pth, training_curves.png, confusion_matrix.png, sample_predictions.png